# Детерминированная замена первых слоёв языковой модели

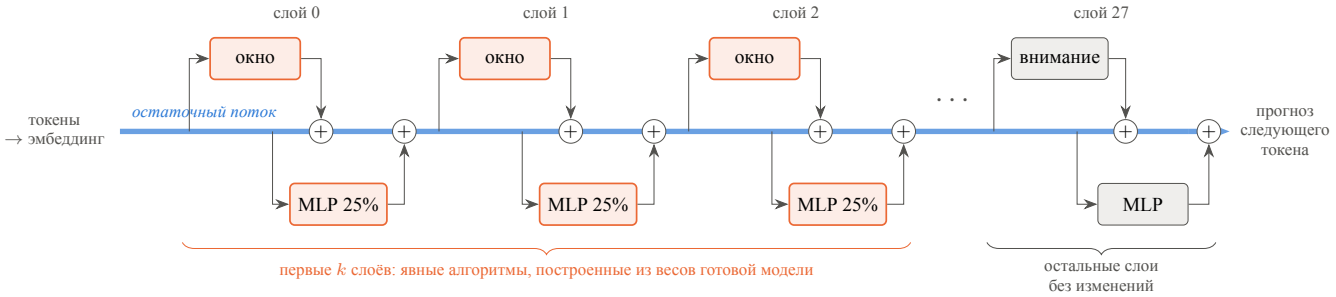
Зачем это нужно, как мы это измеряем и сколько слоёв можно заменить

### Зачем

Языковая модель (здесь — Qwen3-0.6B и Qwen3-1.7B, а также SmolLM2-1.7B семейства Llama) состоит из 28 (24) одинаково устроенных слоёв, и каждый токен текста проходит через все. Первый слой стоит столько же, сколько последний, хотя делает более простую работу: склеивает части слов, смотрит на соседние токены и пунктуацию, запоминает частые сочетания. Мы проверяем, можно ли **первые слои заменить явными дешёвыми алгоритмами**, которые строятся из весов готовой модели и небольшого корпуса текста за минуты — без обучения градиентным спуском и без случайности (отсюда «детерминированная» замена). Если это удаётся, мы получаем:

- **скорость и память** — меньше операций на токен и меньше кэша ключей-значений на длинном контексте;
- **понимание модели** — если слой заменён явной формулой почти без изменения ответов модели, эта формула и есть проверяемое объяснение того, что слой делает;
- **воспроизводимость** — тот же вход даёт тот же выход, а ошибку замены можно измерить и разобрать по слоям.

### Как устроена модель и что именно мы меняем




Текст режется на токены, каждый токен превращается в вектор из 1024 чисел. Дальше этот вектор едет по *остаточному потоку* (синяя линия) через 28 слоёв: каждый слой читает вектор, вычисляет поправку и *прибавляет* её к потоку (знаки «+»). В слое два блока. **Внимание** — единственное место, где токен получает информацию от предыдущих токенов: он сравнивает себя с каждым из них, назначает веса и собирает взвешенную сумму. **MLP** обрабатывает каждый токен отдельно: 3072 «нейронов», каждый проверяет свой признак и, если признак сработал, добавляет к потоку свой вектор. На коротком контексте MLP — около двух третей умножений слоя; внимание дорожает с длиной контекста $T$, потому что сравнивает токен со всеми предыдущими.

Мы берём готовую модель и заменяем первые $k$ слоёв (оранжевые блоки), остальные $28 - k$ не трогаем. Внимание заменяется *окном*: токен смотрит только на 64 предыдущих и на самый первый токен текста (его называют «стоком», модель использует его как «никуда не смотреть»). MLP заменяется подмножеством нейронов: наш метод M-O4 по вектору токена предсказывает, какие 25 % нейронов сработают, и считает только их; SVD-LLM из литературы вместо этого сжимает матрицы нейронов вдвое. Все параметры замены — поправки, предиктор, выбор нейронов — вычисляются по явным формулам (метод наименьших квадратов) из весов модели и 128 тыс. токенов калибровочного текста. Слои заменяются по очереди, и каждая следующая замена калибруется с учётом ошибки предыдущих.

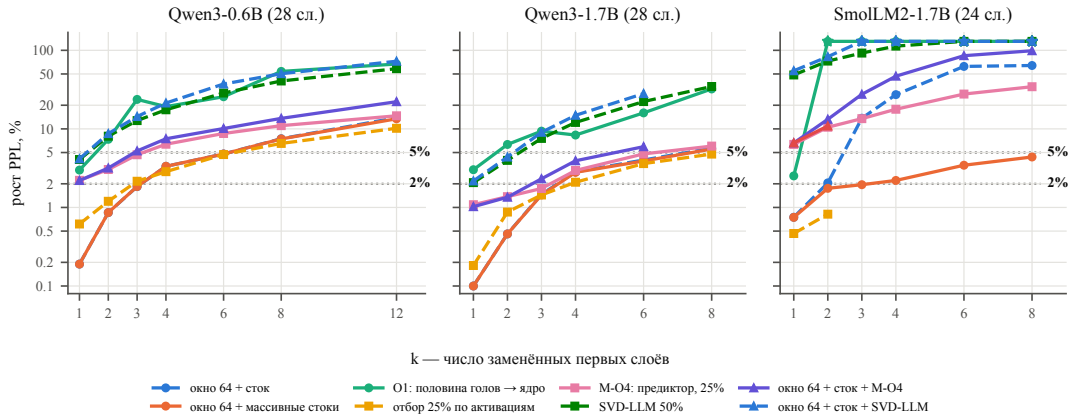
### Как измеряем

- **Данные** — WikiText (Merity и др., 2017; Salesforce Research): статьи английской Википедии категорий Good/Featured. Калибровка — 256 случайных окон по 512 токенов (128 тыс.) из WikiText-103 train (103 млн слов); оценка — первые 32 окна по 1024 токена из WikiText-2 test; наборы не пересекаются. Бенчмарки — из lm-evaluation-harness.
- **Perplexity (PPL)** — «насколько модель удивлена текстом»: экспонента средней ошибки предсказания следующего токена на 32 тыс. токенов WikiText-2. У исходной модели 14.06; меньше — лучше. Главная метрика записки — *рост* PPL после замены. Порог 2 % считаем «практически без потерь» (шум измерения — десятые доли процента), 5 % — «заметно, но приемлемо».
- **KL-расхождение** — насколько распределение вероятностей следующего токена у модели с заменой отличается от исходной модели (0 — совпадает). Точнее PPL: сравнивает с моделью, а не с текстом.
- **Совпадение top-1** — доля позиций, где обе модели называют один и тот же самый вероятный следующий токен.
- **Бенчмарки** — 7 стандартных задач (LAMBADA, HellaSwag, ARC, PIQA, WinoGrande, MMLU: вопросы на знания и здравый смысл) на подвыборках по 200 примеров.
- **Стоимость** — доля умножений на токен во всей модели, которую экономит замена, для контекста $T = 2048$ и 8192, и доля сэкономленного кэша ключей-значений.

### Сколько слоёв можно заменить

| Метод | Что заменено | Сколько слоёв $k$: Qwen3-0.6B / 1.7B / SmolLM2 при $\Delta$PPL$\leq$2% | при $\Delta$PPL$\leq$5% | Экономия умножений (0.6B) при $k_{2\%}$ $T$=2048 / 8192 | при $k_{5\%}$ $T$=2048 / 8192 |
|---|---|---|---|---|---|
| Окно 64 + сток | внимание | 3 / 3 / 1 | 6 / 6 / 2 | 1.7% / 4.6% | 3.3% / 9.3% |
| **Окно 64 + массивные стоки** | внимание | 3 / 3 / 3 | 6 / 6 / 8 | 1.7% / 4.6% | 3.3% / 9.3% |
| **O1: половина голов → позиционное ядро** | внимание | — / — / — | 1 / 1 / 1 | — | 0.1% / 0.4% |
| Отбор 25% нейронов по активациям (Deja Vu) | MLP | 2 / 3 / $\geq$2 | 6 / 8 / $\geq$2 | 0.7% / 0.4% | 2.0% / 1.3% |
| **M-O4: предиктор нейронов, 25%** | MLP | — / 3 / — | 3 / 6 / — | — | 2.4% / 1.6% |
| SVD-LLM, ранг 50% | MLP | — / — / — | 1 / 2 / — | — | 0.7% / 0.4% |
| **Окно 64 + сток + M-O4** | блок | — / 2 / — | 2 / 4 / — | — | 2.7% / 4.2% |
| **Окно 64 + массивные стоки + M-O4** | блок | … / … / — | … / … / — | — | — |
| Окно 64 + сток + SVD-LLM | блок | — / — / — | 1 / 2 / — | — | 1.2% / 2.0% |

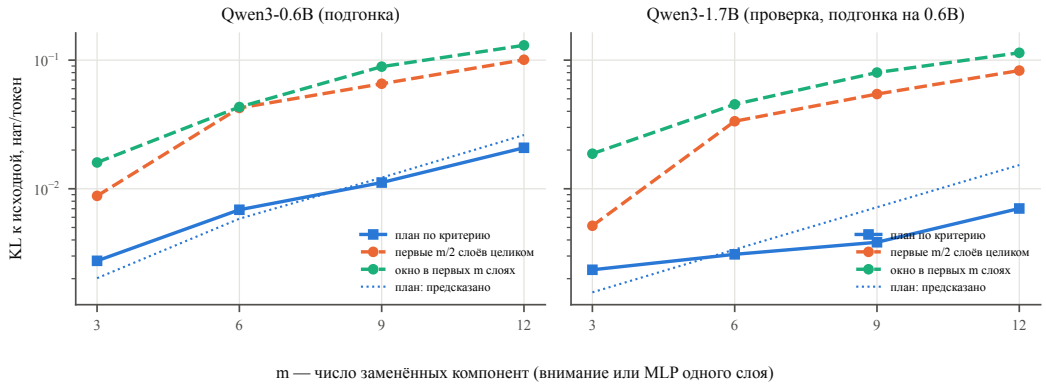
Жирным — наши методы; остальные — из литературы. $k$ — наибольшее число первых слоёв, при котором порог выполняется для всех $k' \leq k$; проверены $k = 1, 2, 3, 4, 6, 8, 12$ (1.7B и SmolLM2 — до 8; «…» — ещё считается; «$\geq k$» — кривая досчитана до $k$ и порог ещё не нарушен). Экономия умножений дана для 0.6B, у 1.7B доли те же с точностью до долей процента.




1. **Внимание первых слоёв заменяется почти даром.** Окно 64 + сток: 3 слоя при росте PPL $\leq 2\,\%$ и 6 при $\leq 5\,\%$; на семи бенчмарках отличия от исходной модели в пределах $\pm 2$ пунктов, т. е. шума. Но экономия умножений мала: 1.7% при $T = 2048$ и 4.6% при $T = 8192$ для $k = 3$ (3.3% и 9.3% для $k = 6$). Главный выигрыш — память: кэш ключей-значений в заменённых слоях сокращается с $T$ до 64 позиций, это 10.6% ($k = 3$) и 21.3% ($k = 6$) всего кэша при $T = 8192$. Ошибка окна растёт плавно: даже при $k = 12$ (почти половина слоёв) PPL выше исходной лишь на 13.9%. На Qwen3-1.7B (тоже 28 слоёв, вдвое шире; посчитано до $k = 8$): окно 64 + сток — 3 слоя при $\leq 2\,\%$ и 6 при $\leq 5\,\%$; M-O4 — 3 и 6; отбор по активациям — 3 и 8.
2. **MLP первых слоёв сжимается дорого.** Лучшее из реализуемого — наш M-O4 (предиктор выбирает 25 % нейронов до их вычисления): $\leq 5\,\%$ до $k = 3$ (рост 4.7% при $k = 3$); SVD-LLM (ранг 50 %) — $k = 1$. Отбор по готовым активациям (Deja Vu: все нейроны вычисляются, слабые отбрасываются перед последней матрицей) держит $\leq 5\,\%$ до $k = 6$, но экономит лишь треть MLP. Комбинации «окно + MLP» наследуют ошибку MLP-части.
3. **Ответ на вопрос.** Без потерь ($\leq 2\,\%$) можно заменить первые 3 слоя внимания; для MLP без потерь — только 2 слоя и только отбором по готовым активациям. С допуском 5 % — 6 слоёв внимания или 2 слоя целиком (окно + M-O4). Выигрыш в скорости при этом — единицы процентов: первые три слоя — это $3/28 \approx 11\,\%$ модели, а внимание на коротком контексте — малая часть слоя. Выиграть в качестве (PPL ниже исходной) не удалось ни одному методу.
4. **Ловушка калибровки.** Наивно откалиброванный M-O4 давал 27.0% при $k = 3$ против 3.0% при $k = 2$ — обрыв на слое, где возникает сток. Причина — 1.6 % окон калибровки (около 4 из 256), где сток возникает не на позиции 0, а на токене вроде «0» или «=»: выход MLP слоя 2 для него имеет норму 7 059 при медиане 4.9, такие строки захватывают метод наименьших квадратов, и ошибка ложится в каналы массивной активации у всех токенов. Управляемый эксперимент: 0, 1, 4, 16 отравленных окон дают 4.2, 4.2, 23.9 и 379 %; Qwen3-1.7B иммунна (1.6–1.7 % при любой дозе). Исключение таких строк снимает обрыв: 4.7% при $k = 3$.

### Что ещё показали эксперименты

**Карта заменяемости и априорный критерий.** Мы заменяли каждый из 28 слоёв *по отдельности* (внимание — окном, MLP — отбором 25 % нейронов) и измеряли KL к исходной модели. Внимание везде заменяется легко, кроме слоя 3, где возникает сток, и последнего слоя; MLP, наоборот, незаменим именно в первых шести слоях и становится заменимым к концу модели. Ошибку замены можно предсказать *до* замены по статистикам самого слоя: доля веса внимания вне окна предсказывает KL окна, локальная ошибка отбора нейронов относительно нормы остаточного потока — KL замены MLP (ранговая корреляция Спирмена 0.91–0.97 на обеих моделях). Критерий подогнан на 0.6B и без изменений применён к 1.7B: план из $m$ замен с наименьшей предсказанной ошибкой даёт KL в 10–15 раз ниже, чем замена первых слоёв подряд (рис. ниже). План сам уводит замену из начала модели в хвост: MLP слоёв 23–27 и внимание слоя 0.




**Почему MLP первых слоёв незаменим.** В слоях 0–5 активны 60–74 % нейронов (в поздних — 40 %), выход MLP составляет 0.45–0.72 нормы остаточного потока (в поздних — 0.2–0.4), а вклад распределён по нейронам равномерно: концентрация вклада в 25 % сильнейших нейронов коррелирует с ошибкой замены на $-0.92$. Ранние MLP пишут большие, плотные и высокоранговые поправки, особенно при сборке слов из кусков токенов; «малое подмножество нейронов» для них не существует.

**Нейроны стока.** Массивную активацию первого токена (норма около 7 000) создают 5 нейронов MLP слоя 2 у 0.6B и 1 нейрон у 1.7B (90–94 % вклада). Удаление их всюду ломает модель: $+54$ % и $+231$ % PPL; удаление на всех позициях, кроме первой, — $+0.3$ % и 0.0 %. Это значит, что любая замена слоя 2 обязана считать первую позицию точно или сохранять эти нейроны явно; в наших методах первая позиция считается точно.

**Обменный курс.** Все работающие замены 0.6B ложатся на одну линию «1 % PPL за 1 % сэкономленных умножений» (контекст 2048); у 1.7B курс в 2.5 раза лучше — около 0.4 % PPL за процент. Методы с ошибочным критерием выбора (SVD-LLM, отбор по косинусной близости состояний, как в ShortGPT) лежат в разы выше линии. Если курс продолжает улучшаться с размером, замена слоёв станет выгодной на моделях 4B–32B — это проверяется на GPU.

**Домен калибровки.** Окно не калибруется и переносится без потерь (1.2 % на pile-10k против 1.8 % на WikiText-2). Калиброванные замены MLP переобучаются под корпус: робастный M-O4 при калибровке на WikiText-103 стоит 4.7 % на WikiText-2 и 16.4 % на pile-10k. Калибровка на смеси доменов выравнивает ошибку (8.6 и 8.0 %); текст, сгенерированный самой моделью, не помогает.

**Второе семейство.** На SmolLM2-1.7B (архитектура Llama, 24 слоя, без QK-нормализации, связанные эмбеддинги) сток привязан не к позиции, а к токену: один нейрон MLP слоя 1 создаёт массивную активацию на слове «the» в любой позиции, и поздние слои смотрят на эти токены как на стоки. Отсюда два правила: при замене MLP нейроны стока сохраняются явно (M-O4 при $k = 2$: 123 % $\to$ 11 %), при замене внимания окно хранит массивные ключи ($k = 8$: 64 % $\to$ 4.4 % при неизменных бенчмарках). Внимание при этом заменяется так же, как у Qwen3 (3 слоя при 2 %, 8 при 5 %), а MLP первых слоёв — заметно хуже (M-O4 6 % уже при $k = 1$); планировщик по критерию переносится (6 компонент: 2.6 % против 16–62 % у префиксов), критерий для MLP — слабее (корреляция $-0.25$ против $-0.91$). Llama-3.2-1B считается.

### Что не работает

- **Быстрое внимание из литературы без переобучения.** Performer (случайные признаки) и линейное внимание ломают модель: нормы ключей в Qwen3 равны 30–60, а эти приближения устойчивы лишь при нормах порядка единицы.
- **Замена голов позиционным ядром (O1).** Даже один слой даёт $+3$ %: в первых слоях внимание зависит от содержания токенов, а не только от расстояния.
- **Окно с «хвостом»** (наш O5: окно плюс грубая сводка далёких токенов) не лучше обычного окна.

### Ограничения

Экономия посчитана по числу умножений: наша реализация окна на чистом PyTorch медленнее встроенного ядра, ускорение требует специального ядра. Калиброванные замены MLP переобучаются под домен калибровки (вне домена ошибка втрое выше), лечится смесью корпусов. Бенчмарки главных столбцов на 1000 примерах ($\pm 1$ пункт), остальных — на 200 ($\pm 2.5$); интервалы для PPL узкие.